

Agentic Computational Reproducibility for Published Research

CSE598 (Agentic AI) — Capstone Project Proposal

Student name	Jason Lunder
Project title	Agentic Computational Reproducibility of Published Research on CORE-Bench
Repository / notebook link	https://github.com/jlunder00/repro-agent
Configuration location	examples/ and .env.example

1 Problem Definition

Published code capsules often fail to run after release because of dependency rot, undocumented environments, or ambiguous run instructions. Verifying a paper’s numbers requires a human to install the toolchain, run the code, and check the output. This project automates that with an agentic system. **Input:** a code capsule (source plus README and/or Dockerfile) and questions about its reported results. **Output:** a `report.json` mapping each question to an answer. **Success:** every answer matches ground truth under the benchmark’s grading rule (§4). **Failure:** any wrong answer, a crash, or no file produced.

2 Motivation and Project Scope

We target CORE-Bench [Siegel et al., 2024b,a]: 270 tasks from 90 papers (computer science, social science, medicine; Python and R), split 45 train / 45 test, each at three tiers: *easy* (a successful run’s output is given, execution forbidden), *medium* (Dockerfile plus README), and *hard* (README only; the agent installs dependencies and infers commands). On the 135-task test split the best published agent (CORE-Agent, GPT-4o) reaches 21.48% on hard [Siegel et al., 2024b]. The capstone goal is an iterative pipeline that improves on this; this proposal covers the baseline it will be measured against. **Scope IN:** Python capsules, CPU-only, small capsules (roughly two-thirds of the test split is under 26 MB). **Scope OUT (deferred):** R capsules, GPU tasks, vision/figure questions, multi-gigabyte capsules, Azure parallel execution.

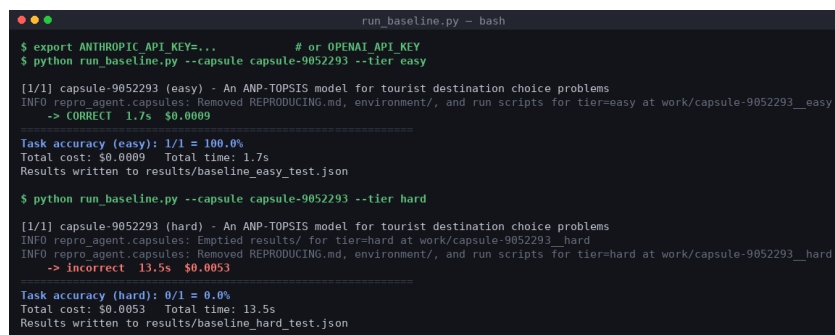
3 Runnable Baseline

The baseline is a single-pass, no-retry pipeline: (1) fetch the capsule, (2) read the README, (3) one LLM call proposing install and run commands, (4) execute those commands once in Docker, (5) one LLM call extracting answers from captured stdout, (6) write `report.json`, (7) score with the vendored official scorer. No retries, self-correction, or re-planning at any stage.

The baseline is the control for the capstone system, which adds an iterative loop that observes failures, revises commands, and re-reads documentation. Holding model, prompts, and one-shot budget fixed and varying only iteration isolates the loop’s marginal value. Single-pass execution attributes each failure to one stage (planning, execution, or extraction). LLM access goes through `litellm`; the pipeline runs unmodified with an Anthropic or OpenAI key.

4 Test Case and Baseline Output

The example is `capsule-9052293` (1.0 MB), an ANP-TOPSIS tourism model. **Input:** the capsule plus the question “Report the closeness coefficient for location L1.” **Expected output:** `report.json` containing 0.844703753651819. All three ground-truth runs are identical, so the prediction interval is a point and an exact match is required. The capsule is run at both tiers.



```
run_baseline.py - bash
$ export ANTHROPIC_API_KEY=... # or OPENAI_API_KEY
$ python run_baseline.py --capsule capsule-9052293 --tier easy

[1/1] capsule-9052293 (easy) - An ANP-TOPSIS model for tourist destination choice problems
INFO repro_agent.capsules: Removed REPRODUCING.md, environment/, and run scripts for tier=easy at work/capsule-9052293_easy
-> CORRECT 1.7s $0.0009

Task accuracy (easy): 1/1 = 100.0%
Total cost: $0.0009 Total time: 1.7s
Results written to results/baseline_easy_test.json

$ python run_baseline.py --capsule capsule-9052293 --tier hard

[1/1] capsule-9052293 (hard) - An ANP-TOPSIS model for tourist destination choice problems
INFO repro_agent.capsules: Removed REPRODUCING.md, environment/, and run scripts for tier=hard at work/capsule-9052293_hard
-> INCORRECT 13.5s $0.0053

Task accuracy (hard): 0/1 = 0.0%
Total cost: $0.0053 Total time: 13.5s
Results written to results/baseline_hard_test.json
```

Figure 1: Baseline output for the same capsule at the easy tier (correct) and the hard tier (incorrect).

Easy: 2/2 correct, \$0.0034 and 3.0s per task.

Hard: 0/2 correct, both failing at execution. The single planning call proposed `pip install openpyxl pandas && python code/script.py`; the run failed and the pipeline reported null. Manual tracing shows the capsule is reproducible with three sequential fixes, each visible only after the previous failure: (i) `ModuleNotFoundError: xlrld`; (ii) `FileNotFoundError`, because the script uses paths relative to `code/`; (iii) `XLRDLError: Excel xlsx file; not supported`, because `xlrld 2.x` dropped `.xlsx` support, requiring `xlrld==1.2.0`. With all three the capsule reproduces 0.844703753651819 exactly.

5 Reproducibility and Run Instructions

`pip install -r requirements.txt`, copy `.env.example` to `.env`, and set one of `ANTHROPIC_API_KEY` or `OPENAI_API_KEY`. Capsule metadata and prompt templates are under `examples/`. Docker is required only for `--tier hard`. `python run_baseline.py --capsule capsule-9052293 --tier easy` fetches the capsule (1.0 MB, cached in `capsules/`), runs the pipeline of §3, and writes results to `results/`. `--list` prints the selection and capsule sizes without downloading or a key; `--subset`, `--tier`, and `--limit` run batches.

6 Initial Evaluation Plan

The iterative system will be compared against this baseline on: (i) official task accuracy (all-or-nothing per task), (ii) per-question accuracy, (iii) wall-clock latency, (iv) dollar cost per task, and (v) failure stage (install, execution, extraction). Published successful tasks averaged \$0.54 versus \$2.59 for failures. The published numbers below (test split, 45 tasks/tier) are the external reference; the benchmark reports no human baseline.

Agent	Model	Easy	Medium	Hard
CORE-Agent	GPT-4o	60.00%	57.78%	21.48%
CORE-Agent	GPT-4o-mini	44.44%	32.59%	16.30%
AutoGPT	GPT-4o	35.56%	37.78%	6.67%
AutoGPT	GPT-4o-mini	8.89%	2.22%	2.22%

Siegel et al. [2024b], Table 5 (pass@1, test split, 45 papers/tier).

Measured baseline (2 Python capsules, `claude-sonnet-5`): easy 2/2 = 100% task and per-question accuracy, \$0.0034 and 3.0 s per task; hard 0/2 = 0%, \$0.0049 and 9.7 s per task, 2/2 failures at execution and none at planning or extraction. The medium tier is not implemented (it needs Docker-in-Docker). The iterative loop will therefore target dependency and invocation repair before answer extraction.

7 Limitations and Next Steps

Dependency rot: as §4 shows, the fix can be an older package version, which up-front planning does not recover. *Sample size*: two capsules is a demonstration, not an estimate. *Scope*: GPU and multi-gigabyte capsules are excluded and R capsules (about half of CORE-Bench) are deferred, so results cover only the Python subset. *Retrieval*: the easy tier concatenates result files in directory order up to a character budget, so an answer late in a long log can be truncated. *Stochastic scoring*: with a 95% prediction interval a faithful reproduction can fall outside by chance, so a perfect agent would not score 100%. Next: extend to the full Python/CPU test subset, implement the medium tier, add the iterative loop targeting execution-stage failures, and evaluate question-guided result retrieval.

References

- Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebel, and Arvind Narayanan. CORE-Bench GitHub repository. <https://github.com/siegelz/core-bench>, 2024a. Accessed 2026-09-06.
- Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebel, and Arvind Narayanan. CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *arXiv preprint arXiv:2409.11363*, 2024b. URL <https://arxiv.org/abs/2409.11363>.